

Block User Service — Full CRUD Documentation

Server: <http://168.144.35.180:3000> Database: ScyllaDB (Live Production)

Status: Active

CREATE Endpoints

1. Block User

Blocks one user from interacting with another user within a specific scope.

Request:

POST <http://168.144.35.180:3000/block-users/blockUser>

Content-Type: application/json

```
{
  "from": "user_A",
  "to": "user_B",
  "scope": "private_chat",
  "reason": "harassment",
  "is_permanent": true
}
```

Parameters:

- `from` (required) — User ID of the person creating the block
- `to` (required) — User ID of the person being blocked
- `scope` (required) — Where the block applies: `private_chat`, `call`, `purchase`, `feed`, `global`
- `reason` (optional) — Text description of why user was blocked
- `is_permanent` (optional) — `true` for permanent, `false` for temporary
- `expires_at` (optional) — Seconds until expiry (for temporary blocks)

Response (201):

```
json
{
  "success": true,
  "result": true
}
```

Database: Writes to `user_blocks` table

2. Block IP Address

Blocks an IP address from accessing the system.

Request:

POST http://168.144.35.180:3000/block-users/blockIP

Content-Type: application/json

```
{
  "ip": "192.168.1.100",
  "reason": "suspicious activity",
  "permanent": true
}
```

Response (200):

json

```
{
  "success": true
}
```

Database: Writes to `system_blocks` table

3. Block Email

Blocks an email address from being used for registration or login.

Request:

POST http://168.144.35.180:3000/block-users/blockEmail

Content-Type: application/json

```
{
  "email": "spam@example.com",
  "reason": "spam account",
  "permanent": true
}
```

Database: Writes to `system_blocks` table (email is stored as SHA-256 hash)

4. Block App Access

Blocks a user from accessing a specific part of the application.

Request:

POST http://168.144.35.180:3000/block-users/blockAppAccess

Content-Type: application/json

```
{
  "userId": "user_A",
  "scope": "marketplace",
  "reason": "policy violation",
  "permanent": true
}
```

Database: Writes to `system_blocks` table

5. Suspend User

Suspends a user account entirely. Admin action.

Request:

POST http://168.144.35.180:3000/block-users/suspendUser

Content-Type: application/json

```
{
  "userId": "user_A",
  "reason": "repeated violations",
  "adminId": "admin_001",
  "flag": "severe",
  "note": "3rd offense"
}
```

Database: Writes to `manual_actions` table

6. Warn User

Issues a warning to a user. Admin action.

Request:

POST http://168.144.35.180:3000/block-users/warnUser

Content-Type: application/json

```
{
  "userId": "user_A",
  "flag": "language",
  "adminId": "admin_001",
  "note": "inappropriate language in chat"
}
```

Database: Writes to `manual_actions` table

READ Endpoints

7. Is User Blocked

Checks if a specific user is blocked by another user. Returns only true or false.

Request:

GET http://168.144.35.180:3000/block-users/isUserBlocked?from=user_A&to=user_B

The `scope` parameter is optional. If omitted, the system checks ALL scopes. If provided, it checks only that specific scope:

GET http://168.144.35.180:3000/block-users/isUserBlocked?from=user_A&to=user_B&scope=private_chat

Response when blocked:

json

```
{
  "blocked": true
}
```

Response when not blocked:

```
json
{
  "blocked": false
}
```

Database: Reads from `user_blocks` table

8. List User Blocks

Lists all user-to-user blocks with pagination and filtering.

Request:

GET `http://168.144.35.180:3000/block-users/listUserBlocks?limit=20`

Optional filters: `from`, `to`, `scope`, `is_permanent`, `flag`, `sort_by`, `sort_order`, `limit`, `nextToken`

Response:

```
json
{
  "items": [
    {
      "blocker_id": "user_A",
      "blocked_id": "user_B",
      "scope": "private_chat",
      "reason": "harassment",
      "is_permanent": 1,
      "created_at": 1779951866000
    }
  ],
  "count": 1,
  "nextToken": null
}
```

Database: Reads from `user_blocks` table

9. Get Blocks For User

Gets ALL block-related data for a specific user across all three tables.

Request:

GET `http://168.144.35.180:3000/block-users/getBlocksForUser?to=user_B`

Response:

```
json
{
  "count": 3,
  "blocks": {
```

```
    "user_blocks": [...],
    "system_blocks": [...],
    "manual_actions": [...]
  }
}
```

Database: Reads from `user_blocks` + `system_blocks` + `manual_actions`

10. Is IP Blocked

Checks if an IP address is blocked.

Request:

GET `http://168.144.35.180:3000/block-users/isIPBlocked?ip=192.168.1.100`

Response:

```
json
{
  "blocked": true,
  "details": { "db": { ... } }
}
```

Database: Reads from `system_blocks` table

11. Is Email Blocked

Checks if an email address is blocked.

Request:

GET `http://168.144.35.180:3000/block-users/isEmailBlocked?email=spam@example.com`

Database: Reads from `system_blocks` table

12. Is App Access Blocked

Checks if a user's app access is blocked for a specific scope.

Request:

GET `http://168.144.35.180:3000/block-users/isAppAccessBlocked?userId=user_A&scope=marketplace`

Database: Reads from `system_blocks` table

13. List System Blocks

Lists all system-level blocks (IP, email, app access).

Request:

GET http://168.144.35.180:3000/block-users/listSystemBlocks?limit=20

Database: Reads from `system_blocks` table

14. Is User Suspended

Checks if a user account is suspended.

Request:

GET http://168.144.35.180:3000/block-users/isUserSuspended?userId=user_A

Database: Reads from `manual_actions` table

15. List Manual Actions

Lists all manual admin actions (suspensions, warnings).

Request:

GET http://168.144.35.180:3000/block-users/listManualActions?limit=20

Database: Reads from `manual_actions` table

DELETE Endpoints

16. Unblock User

Removes a block between two users. This is a soft delete — the record stays in the database but `deleted_at` is set to the current timestamp.

Request:

POST http://168.144.35.180:3000/block-users/unblockUser

Content-Type: application/json

```
{
  "from": "user_A",
  "to": "user_B",
  "scope": "private_chat"
}
```

Response:

json

```
{
  "success": true,
  "result": {
    "blocker_id": "user_A",
    "blocked_id": "user_B",
    "scope": "private_chat",
    "deleted_at": 1779951907000
  }
}
```

Database: Updates `user_blocks` table (sets `deleted_at`)

17. Unsuspend User

Removes a suspension from a user account.

Request:

POST <http://168.144.35.180:3000/block-users/unsuspendUser>

Content-Type: application/json

```
{
  "userId": "user_A"
}
```

Database: Updates `manual_actions` table (sets `deleted_at`)

BATCH / UTILITY Endpoints

18. Batch Check User Blocks

Check multiple user block relationships in one request.

Request:

POST <http://168.144.35.180:3000/block-users/batchCheckUserBlocks>

Content-Type: application/json

```
{
  "blocks": [
    { "from": "user_A", "to": "user_B", "scope": "private_chat" },
    { "from": "user_C", "to": "user_D", "scope": "feed" }
  ]
}
```

Response:

json

```
{
  "results": [
    { "from": "user_A", "to": "user_B", "scope": "private_chat", "blocked": true },
    { "from": "user_C", "to": "user_D", "scope": "feed", "blocked": false }
  ]
}
```

Database: Reads from `user_blocks` table

19. Get User Manual Actions

Gets all manual actions (suspensions, warnings) for a specific user.

Request:

GET http://168.144.35.180:3000/block-users/getUserManualActions?userId=user_A

Response:

json

```
{
```

```

"success": true,
"count": 2,
"items": [
  { "user_id": "user_A", "type": "warning", "flag": "language", "note":
"..."} },
  { "user_id": "user_A", "type": "suspend", "reason": "..." }
]
}

```

Database: Reads from `manual_actions` table

20. User Activity Stats (Risk Score)

Calculates a risk score for a user based on all their blocks, suspensions, and warnings.

Request:

POST `http://168.144.35.180:3000/block-users/UserStats`

Content-Type: `application/json`

```

{
  "userId": "user_A"
}

```

Response:

```

json
{
  "blockScores": {
    "userBlocks": { "private_chat": { "active": 10, "expired": 5 }, "...":
"..."} },
    "systemBlocks": { "app": { "active": 0, "expired": 0 } },
    "manualActions": { "warning": { "active": 5, "expired": 0 },
"suspension": { "active": 0, "expired": 0 } }
  },
  "totalScore": 20,
  "threshold": 500,
  "flagged": false
}

```

Database: Reads from `user_blocks` + `system_blocks` + `manual_actions`

Database Tables

Table	Purpose	Primary Key	Sort Key
<code>user_blocks</code>	User-to-user blocks	<code>blocker_id</code>	<code>sk_scope</code>
<code>system_blocks</code>	IP, email, app blocks	<code>identifier</code>	<code>sk_type</code>
<code>manual_actions</code>	Suspensions, warnings	<code>user_id</code>	<code>sk_ts</code>

All delete operations are **soft deletes** — records are not removed from the database. Instead, the `deleted_at` field is set to the current timestamp. This preserves audit history.